Advanced CPU Architecture and Hardware Accelerators - Lab 5

Maayan Sarig 206511776
Ido Mayerowicz 207180571

Date: 30.06.2025

מבוא

מטרות מעבדה מספר 5:

- תכנון, סינתזה וניתוח של מעבד פשוט תואם *MIPS*
- ארכיטקטורת מעבד *MIPS* צינורית (*Pipelined*) בסקלר, מבוססת על ארכיטקטורת *MIPS* מחזור יחיד (*Single Cycle*).
- הבנה של ההבדל בין מעבד (*CPU*) לבקר מיקרו (*MCU*).
- הבנה של מבנה הזיכרון ב־*FPGA*.

בדיקת ביצועים

בבדיקת הביצועים המרנו את קוד ה- C (4M) הנתון במטלה לקוד אסמבלי.

```c
#define M 4

void addMats(int Mat1[M][M], int Mat2[M][M], int resMat[M][M]){
     define it yourself …
}

void main(){ //int=32bit
     int Mat1[M][M]={{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};
     int Mat2[M][M]={{13,14,15,16},{9,10,11,12},{5,6,7,8},{1,2,3,4}};
     int resMat[M][M];

     addMats(Mat1,Mat2,resMat); // resMat = Mat1 + Mat2
}
```

```
.globl main
.data

SIZE: .word  10
arr1: .word  1 ,2 ,3 ,4 ,5 ,6 ,7 ,8 ,9 ,10,
arr2: .word  100,99,98,97,96,95,94,93,92,91,

res1: .space 40  # SIZE*4=40[Byte] - ADD result array
res2: .space 40  # SIZE*4=40[Byte] - MUL result array
res4: .space 40  # SIZE*4=40[Byte] - SUB result array

.text
main:
        la $s0,SIZE     # s0 points to SIZE
        lw $s0,0($s0)   # s0 = Mem[s0] = SIZE
        la $t1,arr1     # t1 points to arr1
        la $t2,arr2     # t2 points to arr2
        la $s1,res1     # s1 points to res1
        la $s2,res2     # s2 points to res2
        la $s4,res4     # s4 points to res4
loop:
        lw $t3, 0($t1)  # t3 = arr1[i] = Mem[t0]
        lw $t4, 0($t2)  # t4 = arr2[i] = Mem[t1]
        add $t5,$t3,$t4
        sw  $t5,0($s1)  # Mem[s1] = res1[i]= arr1[i]+arr2[i]
        mul $t5,$t3,$t4
        sw  $t5,0($s2)  # Mem[s2] = res2[i]= arr1[i]*arr2[i]
        sub $t5,$t3,$t4
        sw  $t5,0($s4)  # Mem[s4] = res4[i]= arr1[i]-arr2[i]

        addi $t1,$t1,4
        addi $t2,$t2,4
        addi $s1,$s1,4
        addi $s2,$s2,4
        addi $s4,$s4,4
        addi $s0,$s0,-1
        bne  $s0,$zero,loop

finish: beq $zero,$zero,finish
```

לאחר מכן המרנו את קוד האסמבלי לקבצי ITCM ו- DTCM על מנת לבצע עליהם סימולציה ולבסוף צריבה, סינתזה ובדיקות על גבי ערכת ה FPGA.

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+C)	Value (+10)	Value (+14)	Value (+18)	Value (+1C)
0x00000000	0x0000000a	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x00000006	0x00000007
0x00000020	0x00000008	0x00000009	0x0000000a	0x00000064	0x00000063	0x00000062	0x00000061	0x00000060
0x00000040	0x0000005f	0x0000005e	0x0000005d	0x0000005c	0x0000005b	0x00000065	0x00000065	0x00000065
0x00000060	0x00000065	0x00000065	0x00000065	0x00000065	0x00000065	0x00000065	0x00000065	0x00000064
0x00000080	0x000000c6	0x00000126	0x00000184	0x000001e0	0x0000023a	0x00000292	0x000002e8	0x0000033c
0x000000a0	0x0000038e	0xffffff9d	0xffffff9f	0xffffffa1	0xffffffa3	0xffffffa5	0xffffffa7	0xffffffa9
0x000000c0	0xffffffab	0xffffffad	0xffffffaf	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000000e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000001a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000001c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000001e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

00000057	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0000004f	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000047	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0000003f	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000037	00000000	00000000	00000000	00000000	00000000	FFFFFFAF	FFFFFFAD	FFFFFFAB
0000002f	FFFFFFA9	FFFFFFA7	FFFFFFA5	FFFFFFA3	FFFFFFA1	FFFFFF9F	FFFFFF9D	0000038E
00000027	0000033C	000002E8	00000292	0000023A	000001E0	00000184	00000126	000000C6
0000001f	00000064	00000065	00000065	00000065	00000065	00000065	00000065	00000065
00000017	00000065	00000065	00000065	0000005B	0000005C	0000005D	0000005E	0000005F
0000000f	00000060	00000061	00000062	00000063	00000064	0000000A	00000009	00000008
	00000060	00000061	00000062	00000063	00000064	0000000A	00000009	00000008
	00000007	00000006	00000005	00000004	00000003	00000002	00000001	0000000A

נראה כי קיימת זהות מוחלטת בין זיכרון הסימולציה ובין זיכרון ה- Mars לאחר הריצה.

<u>מבנה המערכת</u>

קיבלנו מעבד single cycle והתבקשנו להוסיף לו פקודות ולאחר מכן להמיר אותו למעבד pipeline לפי המבנה הסכמטי הבא:

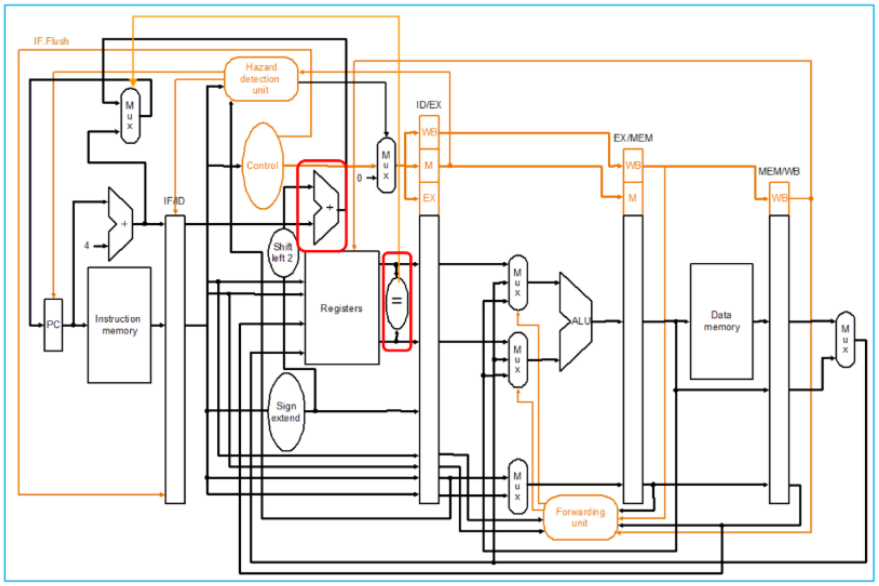
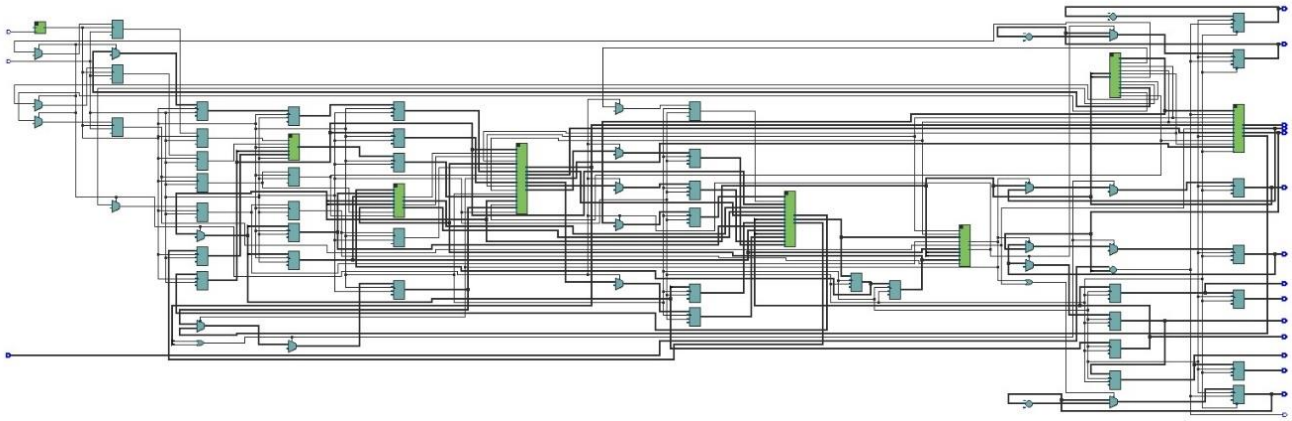


Figure 6: Five-stage pipelined MIPS architecture with forwarding and single delay slot support

במשימה נדרשנו לחשב את ה IPC של מעבד בתצורת ארכיטקטורה של pipeline ולטובת כך נשתמש במניית מחזורי השעון של המעבד ובמונה ההוראות של מערכת ה - MARS.

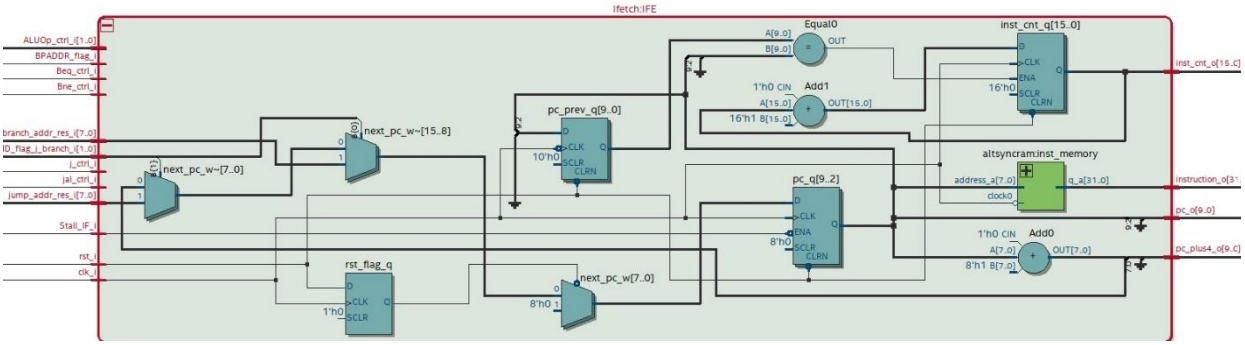
<u>מימוש RTL של כל מודול במערכת</u>

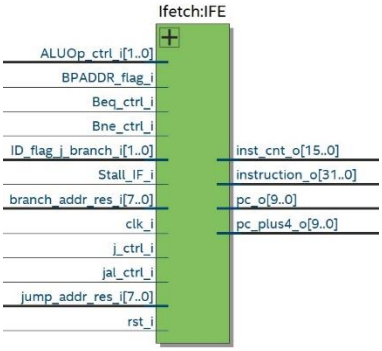
<u>RTL של כלל המערכת</u>



<u>Fetch</u>

יחידה האחראית על ה PC, מבצעת שליפה של הוראות ה ITCM מתוך זכרון הפקודות.



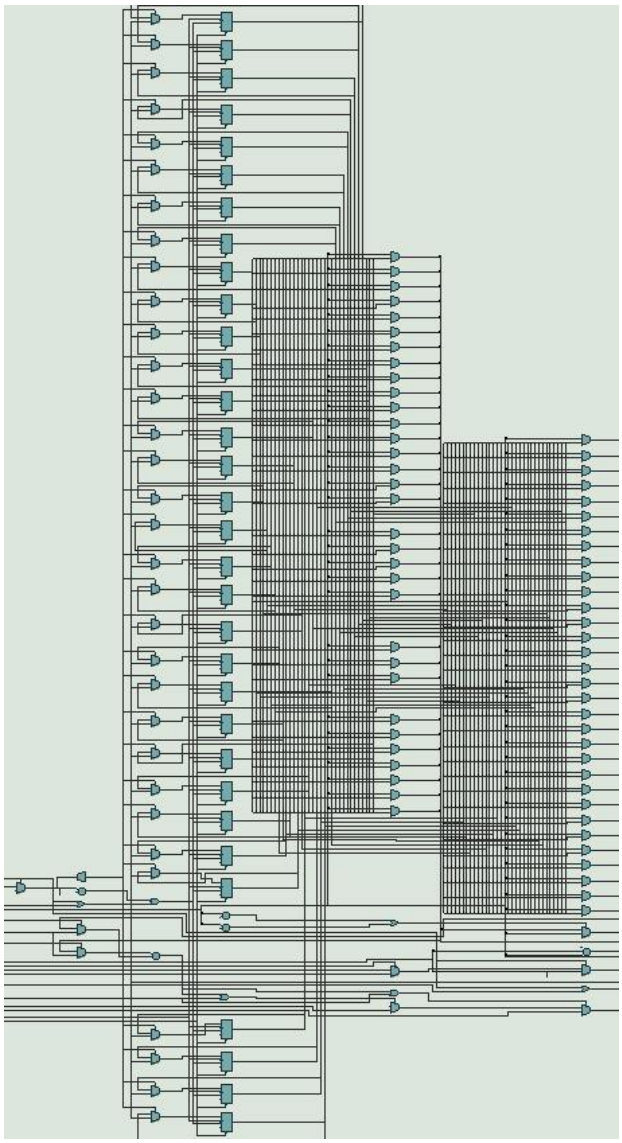


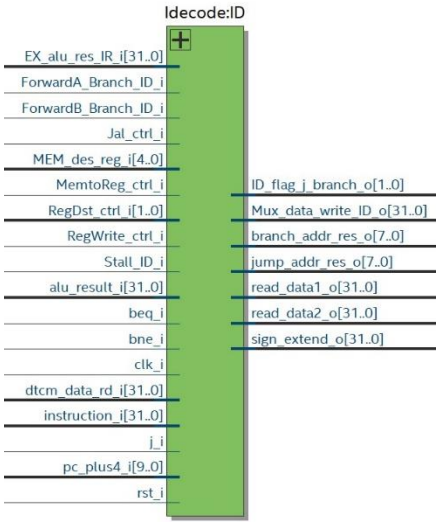
Decode

יחידה המקבלת הוראה ואותות ממערכת הבקרה ובהתאם אליהם מעבירה את האותות הרצויים לפי ה ISA לפי הקידוד הבא:

Type	-31-			format (bits)		-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

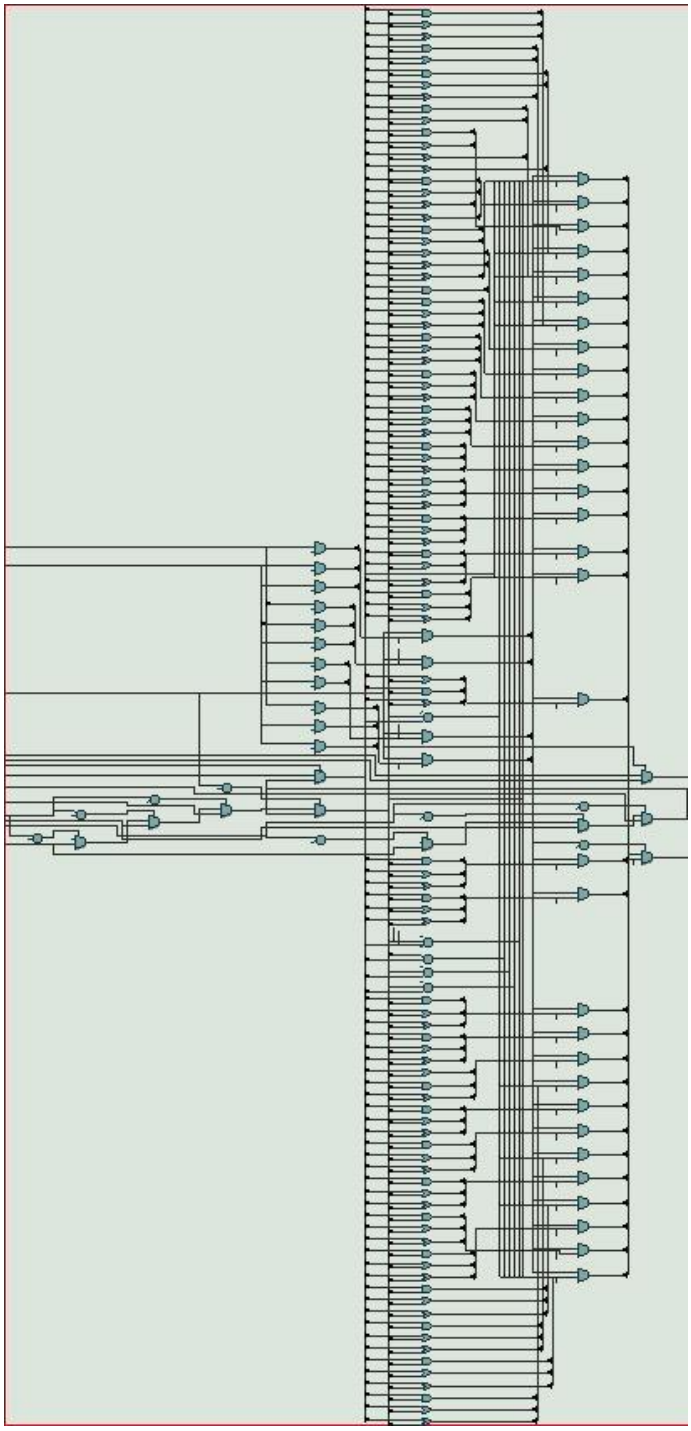
ביחידה זו קיים ה Register File אשר אחראי על שמירת ערכי הרגיסטרים של המעבד, כמו כן יחידה זו אחראית על שליפה ושמירה של ערכים בתוך המעבד.

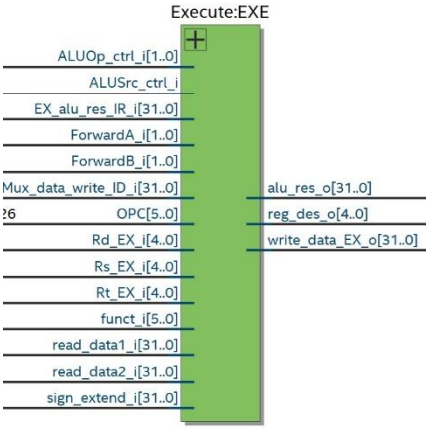




Execute

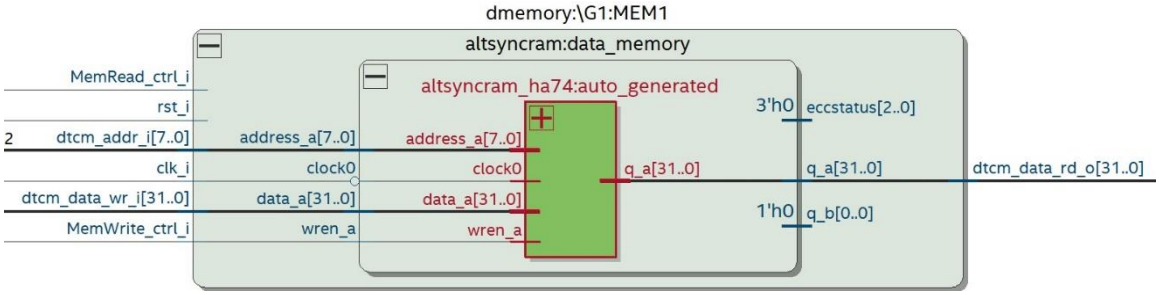
יחידה המקבלת ערכים לביצוע פעולות אריתמטיות שונות, הנקבעות ע"פ מערכת הבקרה. יחידה זו לוקחת חלק בביצוע של פעולות מסוג Rtype ו- Itype בלבד.

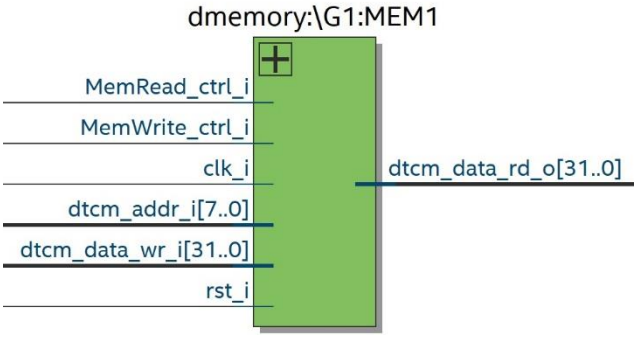




Memory

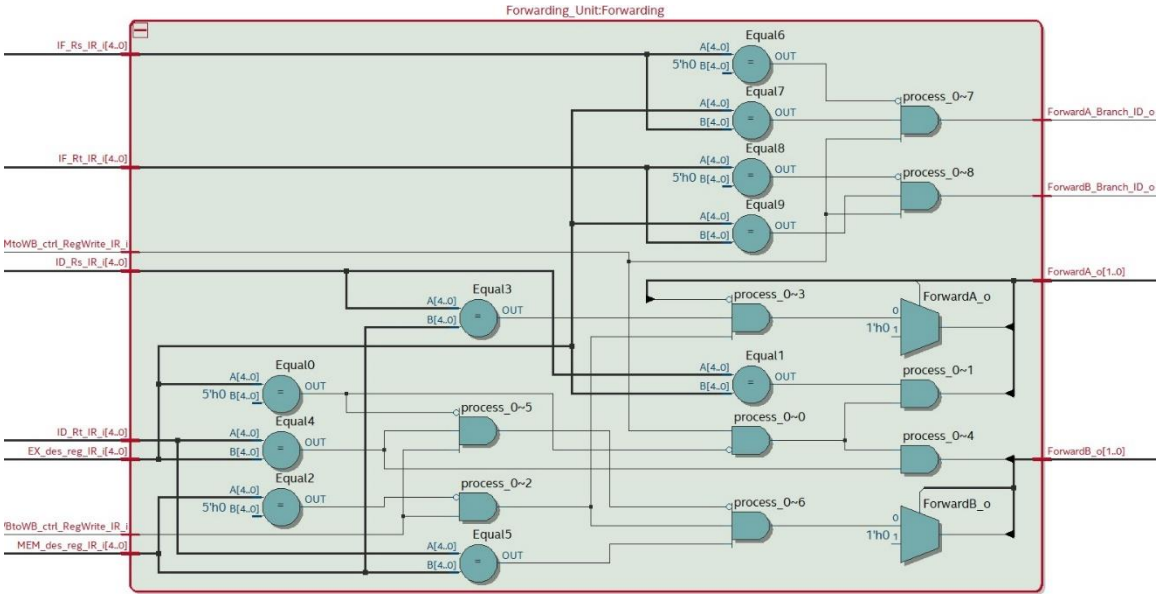
יחידה זכרון, מקבל כתובת ולעיתים גם מידע, ולפי אותות הבקרה שולפת/מאחסנת ערכים.

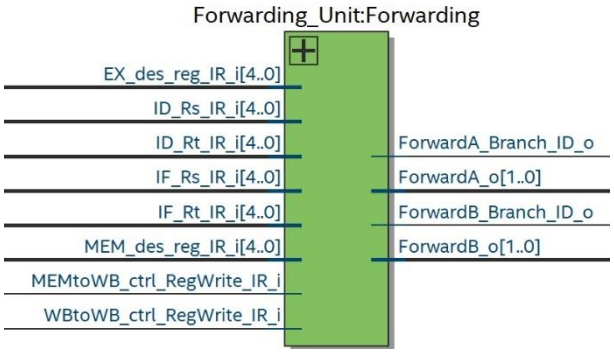




Forwarding Unit

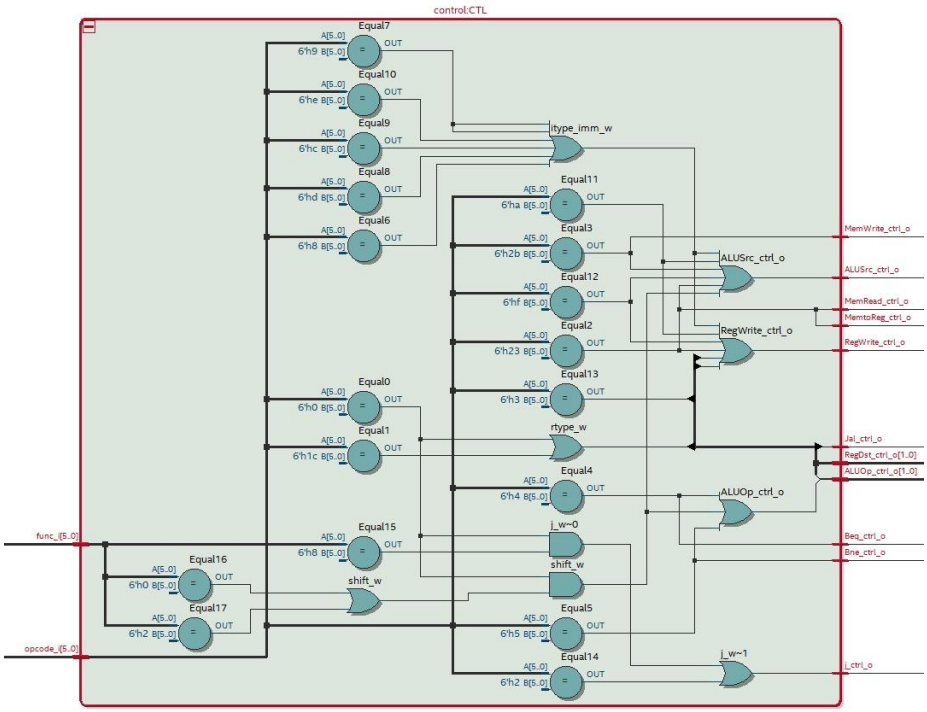
יחידה שמטרתה להאיץ חומרה ולפתור בעיות שנובעות מאריכטקטורת ה pipeline, מקבלת מידע מכלל היחידות ובהתאם לאותות הבקרה מבצעת קידום של מידע משלב ה memory וה- writeback אל ה- decode/execute.

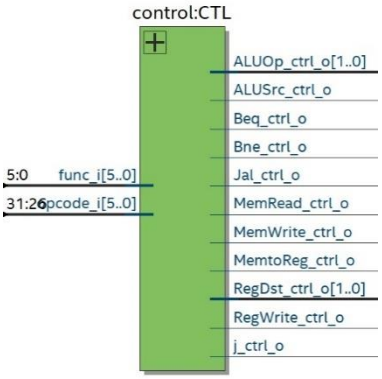




Control

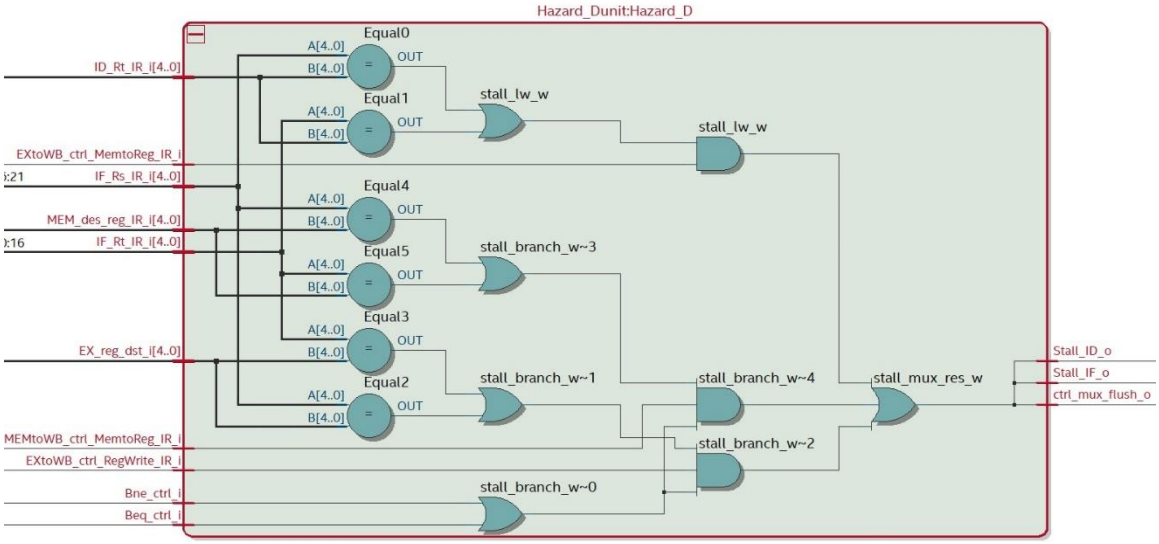
יחידה שמקבלת את ההוראה, מפענחת אותה לפי משפחת פקודה, מספר פקודה ופונקציה ומדליקה אותות בקרה לכלל המערכת בהתאם. אותות הבקרה הם אלו האחראיים על זרימת הנתונים במערכת.

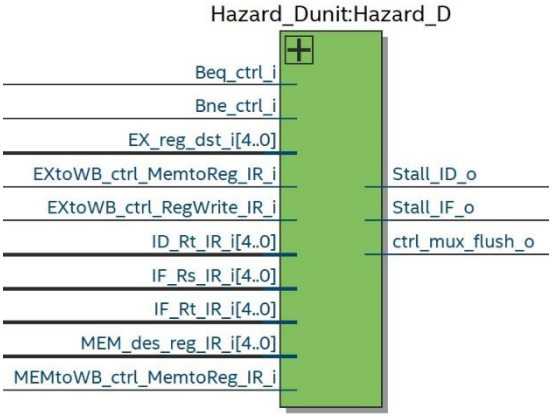




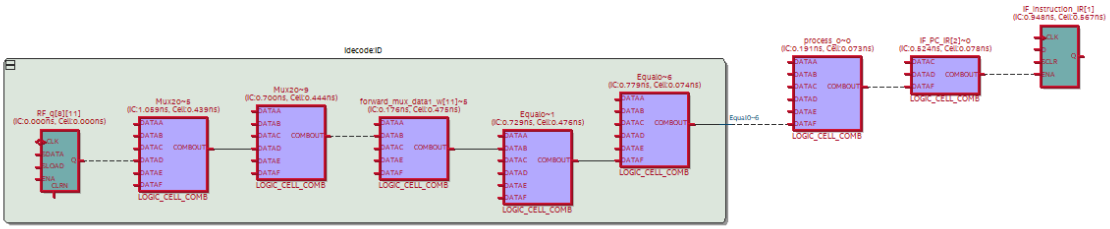
Hazard Detection Unit

יחידה המזהה סכנות צפויות בקוד (ביצוע הוראות אשר משפיעות זו על זו ופוגמות בביצוע הפקודות כראוי), המערכת מבצעת עיכוב (stall) או הדחה (הכנסה של אפסים למערכת, flush) על מנת לפתור את הבעיות הללו.





נתיב קריטי



נבחין כי הנתיב הקריטי נמצא בין יחידות IDecode ליחידת IFetch דבר שעולה בקנה אחד עם ההנחה שהמסלול הקריטי צריך עבור דרך שלב הפענוח, שהיא לוגיקה טורית ארוכה. לאחר מכן, הוא מוזן חזרה דרך רכיבים צירופיים לתוך ה-IFetch מה שיוצר מסלול ארוך לאות שמתבטא בפרק זמן ארוך.

תדר מקסימלי (ע"פ נתיב קריטי)

Slow 1100mV 85C Model Fmax Summary

<<Filter>>

	Fmax	Restricted Fmax	Clock Name	Note
1	100.29 MHz	100.29 MHz	altera_reserved_tck	

חישוב STdept

$$\text{STdept} = \frac{\text{Embedded memory size} - \text{Desgin usage memory size}}{\#STchanels}$$
$$= \frac{5{,}662{,}720bits - 290{,}816bits}{268bits} = 20{,}044$$

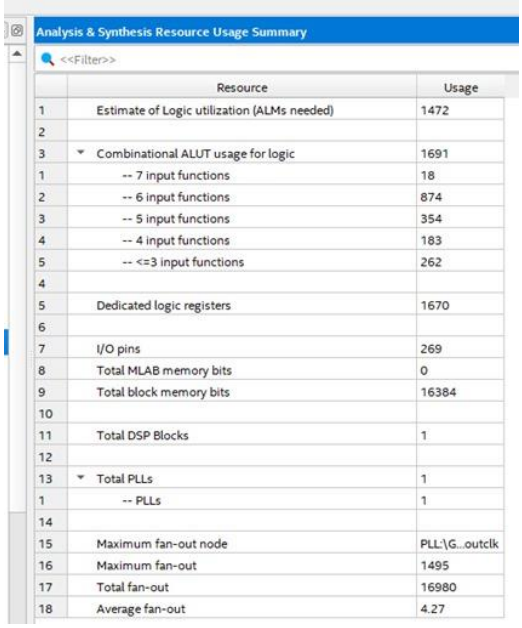
חישוב IPC

נחשב את ה IPC ע"י הנוסחה הבאה:

$$IPC = \frac{INSCNT_O - dept * FHCNT_O}{CKLCNT_O} = \frac{MARSInstructionCounter}{CLKCNT_o} = \frac{181}{238} = 76.05\%$$

מונה ההוראות של ה MARS עבור התוכנית:

שימוש בלוגיקה ($Logic\ Usage$)



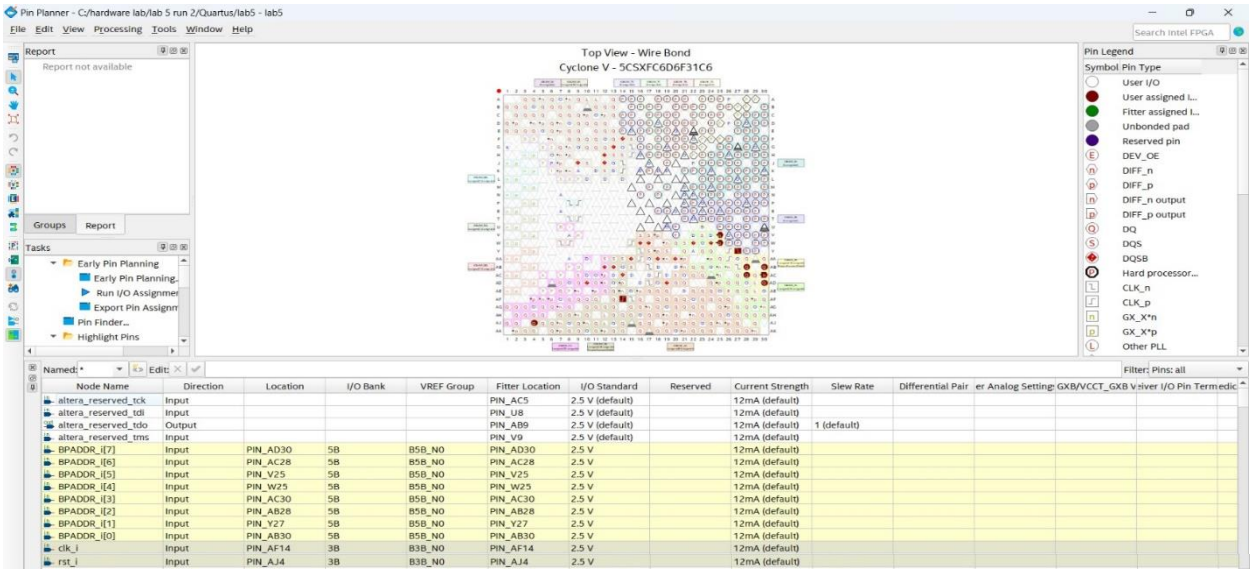
Analysis & Synthesis Resource Usage Summary

<<Filter>>

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	1472
2		
3	Combinational ALUT usage for logic	1691
1	-- 7 input functions	18
2	-- 6 input functions	874
3	-- 5 input functions	354
4	-- 4 input functions	183
5	-- <=3 input functions	262
4		
5	Dedicated logic registers	1670
6		
7	I/O pins	269
8	Total MLAB memory bits	0
9	Total block memory bits	16384
10		
11	Total DSP Blocks	1
12		
13	Total PLLs	1
1	-- PLLs	1
14		
15	Maximum fan-out node	PLL:\G...outclk
16	Maximum fan-out	1495
17	Total fan-out	16980
18	Average fan-out	4.27

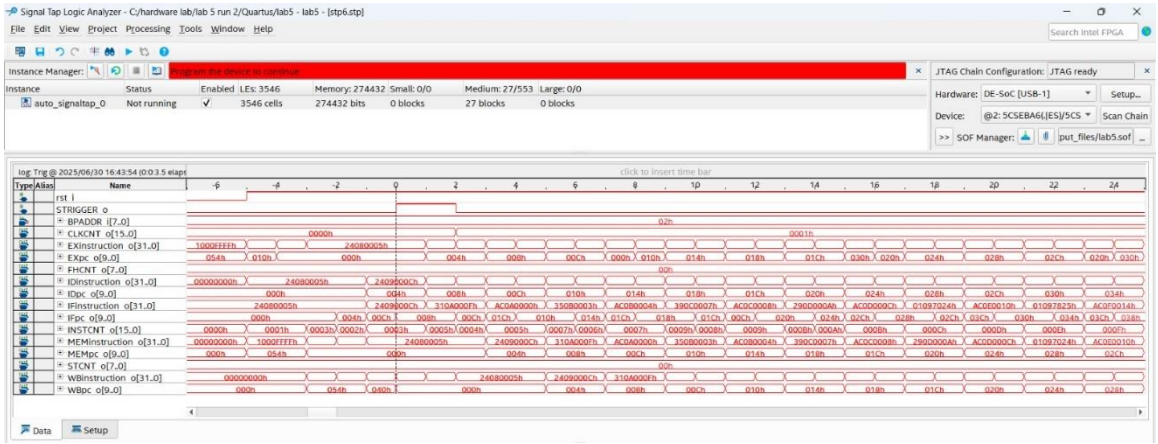
FPGA

במעבדה זו השתמשנו בערכה $DE10 - Standard$, הקצאת הפינים תתבצע ע"פ הוראות היצרן ומצורפת תמונה למחשה:



Signal Tap

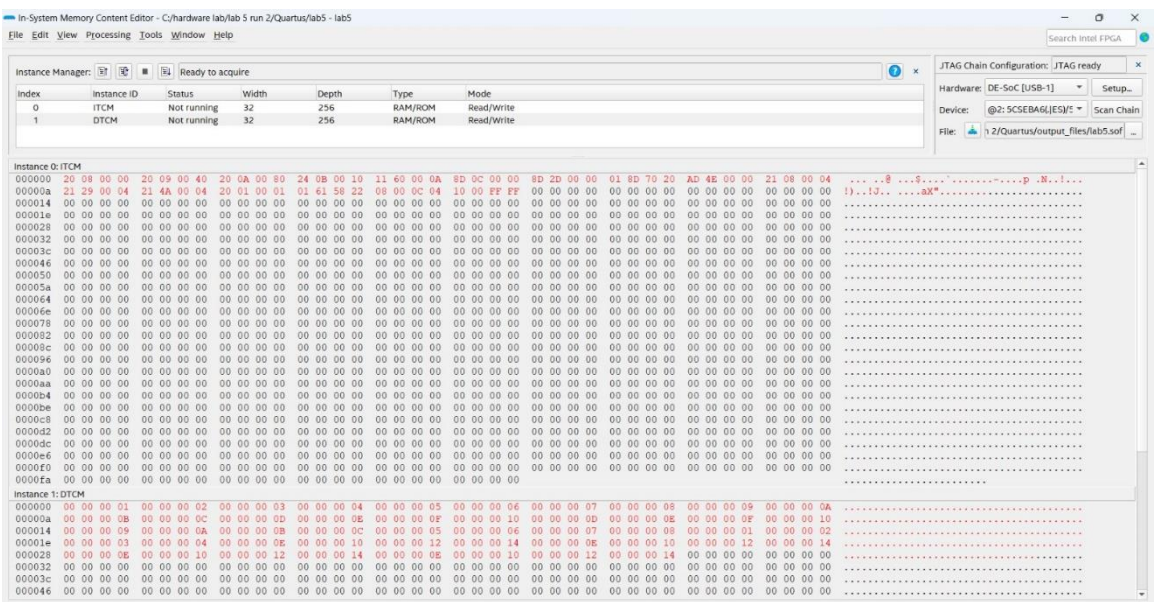
התבקשנו לבצע וריפיקציה של החומרה בעזרת ה Signal Tap, ניעזר ב Strigger שהינו פלט של המערכת, כך שבעליה שלו לערך '1' (ב- rising edge) ה- Signal Tap יתחיל לקרוא ערכים מהמערכת ולהכניס אותם לרגיסטר הנע.



trigger: 2025/06/30 16:43:35 #0 | Lock mode: Allow all changes

Node			Data Enable	Trigger Enable	Trigger Conditions
Type	Alias	Name	268	268	1 Basic AN
		rst_i	✓	✓	
		STRIGGER_o	✓	✓	
		BPADDR_i[7..0]	✓	✓	XXh
		CLKCNT_o[15..0]	✓	✓	XXXXh
		EXinstruction_o[31..0]	✓	✓	XXXXXXXXh
		EXpc_o[9..0]	✓	✓	XXXh
		FHCNT_o[7..0]	✓	✓	XXh
		IDinstruction_o[31..0]	✓	✓	XXXXXXXXh
		IDpc_o[9..0]	✓	✓	XXXh
		IFinstruction_o[31..0]	✓	✓	XXXXXXXXh
		IFpc_o[9..0]	✓	✓	XXXh
		INSTCNT_o[15..0]	✓	✓	XXXXh
		MEMinstruction_o[31..0]	✓	✓	XXXXXXXXh
		MEMpc_o[9..0]	✓	✓	XXXh
		STCNT_o[7..0]	✓	✓	XXh
		WBinstruction_o[31..0]	✓	✓	XXXXXXXXh
		WBpc_o[9..0]	✓	✓	XXXh

בדיקת הרצה לתוכנית המבוקשת (M4):



Text Segment

Bkpt	Address	Code	Basic	Source
	0x00003000	0x20080000	addi $8,$0,0x00000000	11: la $t0, Mat1
	0x00003004	0x20090040	addi $9,$0,0x00000040	12: la $t1, Mat2
	0x00003008	0x200a0080	addi $10,$0,0x00000080	13: la $t2, resMat
	0x0000300c	0x240b0010	addiu $11,$0,0x0000...	15: li $t3, 16
	0x00003010	0x1160000a	beq $11,$0,0x0000000a	17: beq $t3, $zero, done
	0x00003014	0x8d0c0000	lw $12,0x00000000($8)	19: lw $t4, 0($t0)
	0x00003018	0x8d2d0000	lw $13,0x00000000($9)	20: lw $t5, 0($t1)
	0x0000301c	0x018d7020	add $14,$12,$13	21: add $t6, $t4, $t5
	0x00003020	0xad4e0000	sw $14,0x00000000($10)	22: sw $t6, 0($t2)
	0x00003024	0x21080004	addi $8,$8,0x00000004	24: addi $t0, $t0, 4
	0x00003028	0x21290004	addi $9,$9,0x00000004	25: addi $t1, $t1, 4
	0x0000302c	0x214a0004	addi $10,$10,0x0000...	26: addi $t2, $t2, 4
	0x00003030	0x20010001	addi $1,$0,0x00000001	27: subi $t3, $t3, 1
	0x00003034	0x01615822	sub $11,$11,$1	
	0x00003038	0x08000c04	j 0x00003010	28: j loop
✔	0x0000303c	0x1000ffff	beq $0,$0,0xffffffff	31: beq $zero, $zero, done

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+C)	Value (+10)	Value (+14)	Value (+18)	Value (+1C)
0x00000000	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x00000006	0x00000007	0x00000008
0x00000020	0x00000009	0x0000000a	0x0000000b	0x0000000c	0x0000000d	0x0000000e	0x0000000f	0x00000010
0x00000040	0x0000000d	0x0000000e	0x0000000f	0x00000010	0x00000009	0x0000000a	0x0000000b	0x0000000c
0x00000060	0x00000005	0x00000006	0x00000007	0x00000008	0x00000001	0x00000002	0x00000003	0x00000004
0x00000080	0x0000000e	0x00000010	0x00000012	0x00000014	0x0000000e	0x00000010	0x00000012	0x00000014
0x000000a0	0x0000000e	0x00000010	0x00000012	0x00000014	0x0000000e	0x00000010	0x00000012	0x00000014
0x000000c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000000e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000001a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

נראה כי קיימת זהות מוחלטת בין זיכרון הסימולציה ובין זיכרון ה- Mars לאחר הריצה.

<u>נוספים:</u>

רשמנו שתי תוכניות נוספות על התוכנית הנתונה, לבדיקת המעבד, נראה כי עברנו ובדקנו את כלל הפקודות שהוגדרו למעבד ובנוסף נוכל לראות כי הפלט של מערכת ה- Mars זהה למערכת שלנו.

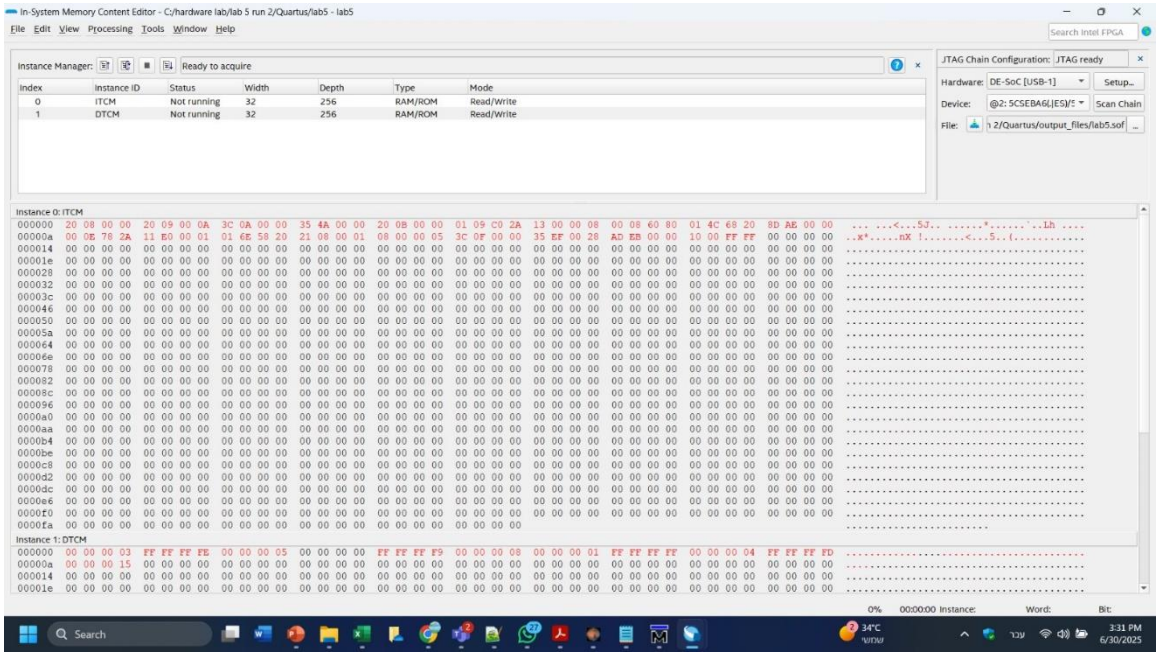
טסט מספר 5:

תיאור של הקוד בשפת C:

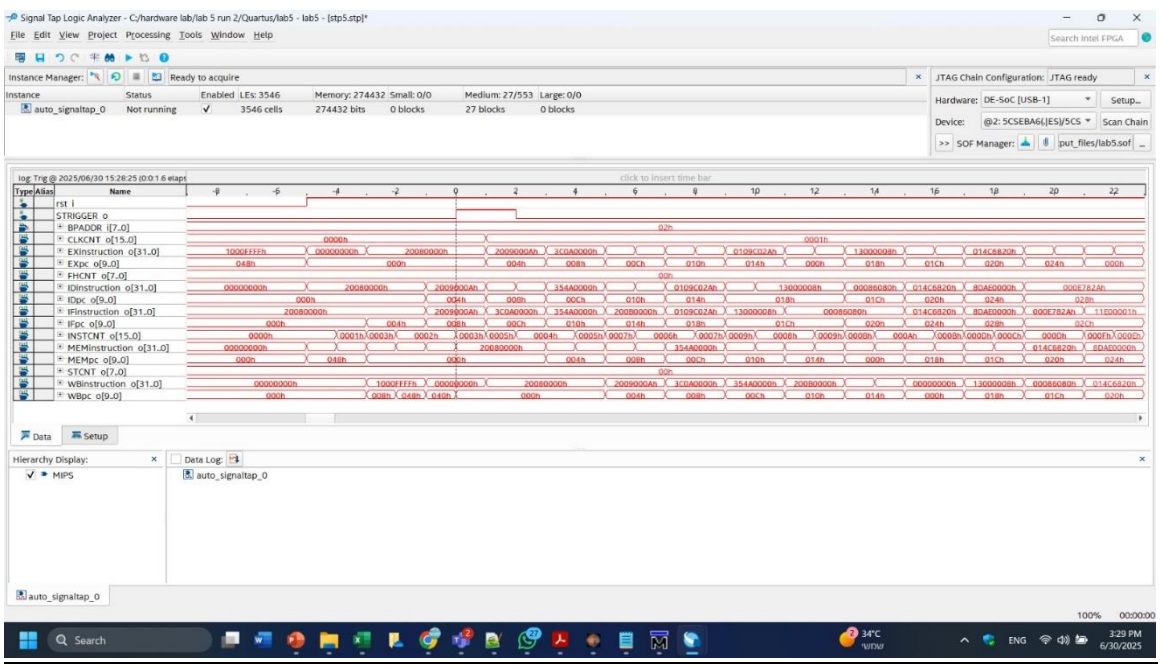
```c

int arr[10] = {3, -2, 5, 0, -7, 8, 1, -1, 4, -3};
int sum = 0;
for (int i = 0; i < 10; i++
{
    if (arr[i] > 0)
        sum += arr[i];
}
```

הקוד בשפת אסמבלי:

```
.data
arr:    .word 3, -2, 5, 0, -7, 8, 1, -1, 4, -3
sum:    .word 0

    .text
    .globl main
main:
    li $t0, 0           # i = 0
    li $t1, 10          # limit = 10
    la $t2, arr         # $t2 = &arr[0]
    li $t3, 0           # $t3 = sum

loop:
    slt $t8, $t0, $t1       # if i < 10
    beq $t8, $zero, end_loop

    sll $t4, $t0, 2         # offset = i * 4
    add $t5, $t2, $t4       # $t5 = &arr[i]
    lw  $t6, 0($t5)         # $t6 = arr[i]

    slt $t7, $zero, $t6     # if 0 < arr[i]
    beq $t7, $zero, skip    # skip if not positive

    add $t3, $t3, $t6       # sum += arr[i]

skip:
    addi $t0, $t0, 1        # i++
    j loop

end_loop:
    la $t7, sum
    sw $t3, 0($t7)          # sum = total

forever:
    beq $zero, $zero, forever # infinite loop
```


In-System Memory Content Editor - C:/hardware lab/lab 5 run 2/Quartus/lab5 - lab5
File Edit View Processing Tools Window Help
Instance Manager: Ready to acquire
Index Instance ID Status Width Depth Type Mode
0 ITCM Not running 32 256 RAM/ROM Read/Write
1 DTCM Not running 32 256 RAM/ROM Read/Write
JTAG Chain Configuration: JTAG ready
Hardware: DE-SoC [USB-1]
Device: @2: 5CSEBA6(JES)/5
Scan Chain
Instance 0: ITCM
Instance 1: DTCM

:Signal Tap


Signal Tap Logic Analyzer - C:/hardware lab/lab 5 run 2/Quartus/lab5 - lab5 - [stp5.stp]*
File Edit View Project Processing Tools Window Help
Instance Manager: Ready to acquire
Instance Status Enabled LEs: 3546 Memory: 274432 Small: 0/0 Medium: 27/553 Large: 0/0
auto_signaltap_0 Not running 3546 cells 274432 bits 0 blocks 27 blocks 0 blocks
JTAG Chain Configuration: JTAG ready
Hardware: DE-SoC [USB-1]
Device: @2: 5CSEBA6(JES)/5CS
Scan Chain
Type Alias Name
rst_i
STRIGGER_o
BPADDR_i[7..0]
CLKCNT_o[15..0]
EXinstruction_o[31..0]
EXpc_o[9..0]
FHCNT_o[7..0]
IDinstruction_o[31..0]
IDpc_o[9..0]
IFinstruction_o[31..0]
IFpc_o[9..0]
INSTCNT_o[15..0]
MEMinstruction_o[31..0]
MEMpc_o[9..0]
STCNT_o[7..0]
WBinstruction_o[31..0]
WBpc_o[9..0]
Data
Setup
Hierarchy Display:
MIPS
Data Log:
auto_signaltap_0

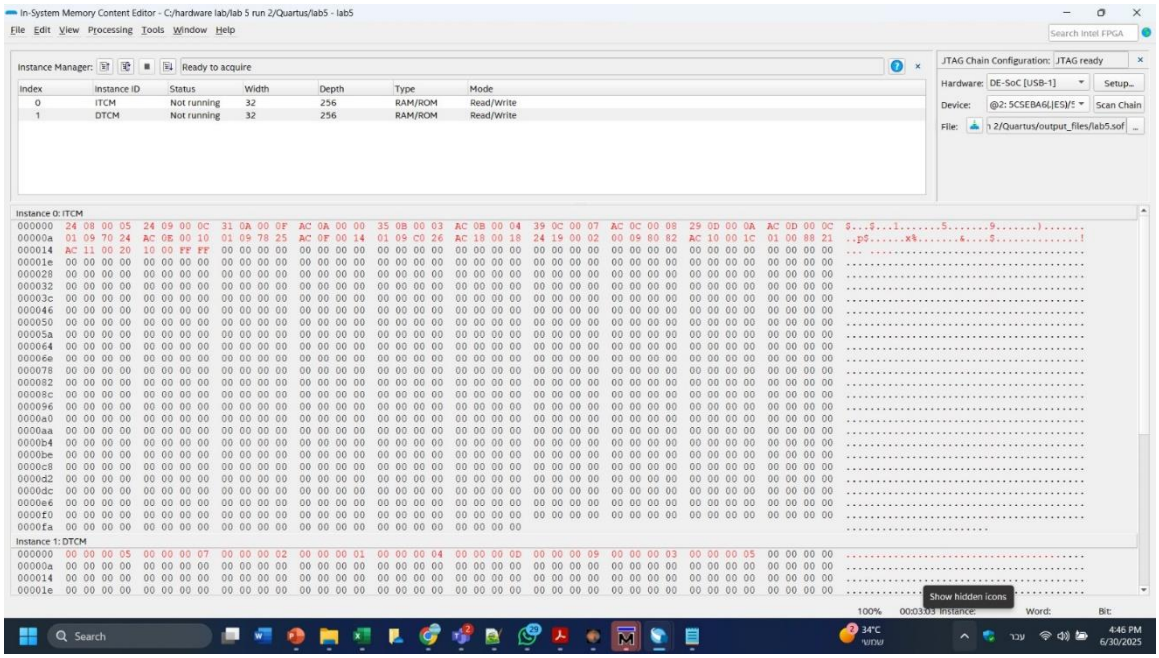
טסט מספר 6:

תיאור של הקוד בשפת C:

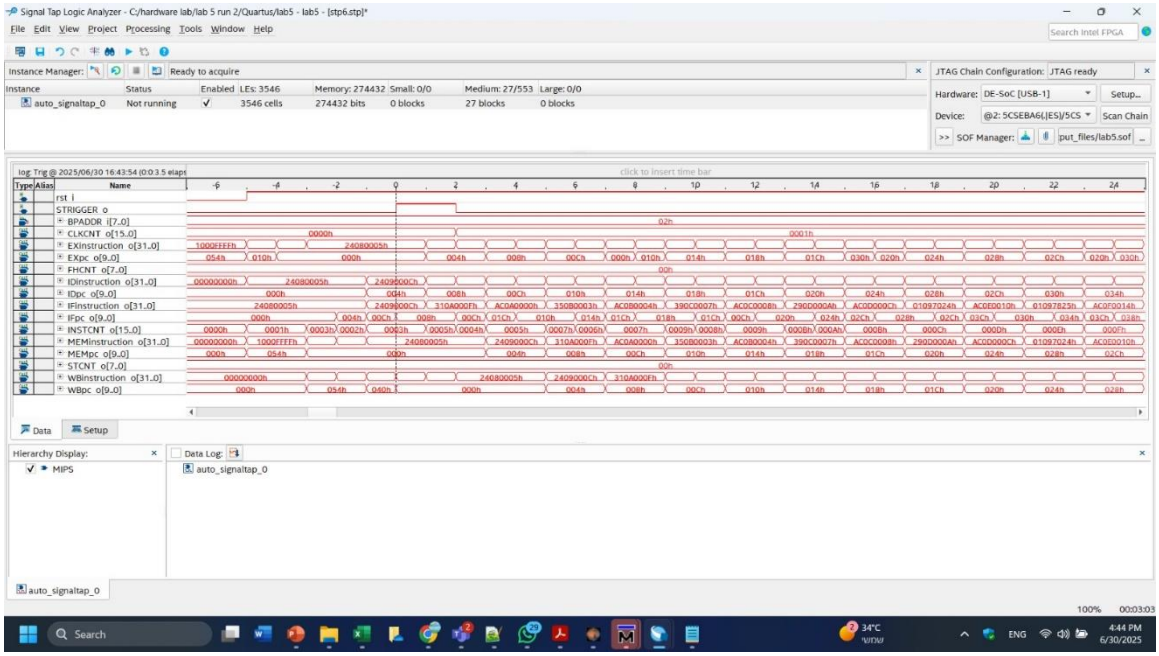
```c
int a = 5;
int b = 12;
int res1 = a & 0xF;          // andi
int res2 = a | 0x3;          // ori
int res3 = a ^ 0x7;          // xori
int res4 = (a < 10);         // slti
int res5 = a & b;            // and
int res6 = a | b;            // or
int res7 = a ^ b;            // xor
int res8 = b >> 2;           // srl
int res9 = a;                // move
```

הקוד בשפת אסמבלי:

```asm
.data
res1: .word 0
res2: .word 0
res3: .word 0
res4: .word 0
res5: .word 0
res6: .word 0
res7: .word 0
res8: .word 0
res9: .word 0

.text
.globl main
main:
    # a = 5
    li $t0, 5          # $t0 = a
    # b = 12
    li $t1, 12         # $t1 = b

    # res1 = a & 0xF
    andi $t2, $t0, 0xF
    sw   $t2, res1

    # res2 = a | 0x3
    ori  $t3, $t0, 0x3
    sw   $t3, res2

    # res3 = a ^ 0x7
    xori $t4, $t0, 0x7
    sw   $t4, res3

    # res4 = (a < 10)
    slti $t5, $t0, 10
    sw   $t5, res4

    # res5 = a & b
    and  $t6, $t0, $t1
    sw   $t6, res5

    # res6 = a | b
    or   $t7, $t0, $t1
    sw   $t7, res6

    # res7 = a ^ b
    xor  $t8, $t0, $t1
    sw   $t8, res7

    # res8 = b >> 2
    li   $t9, 2
    srl  $s0, $t1, 2
    sw   $s0, res8

    # res9 = a  (move דרך addu)
    addu $s1, $t0, $zero
    sw   $s1, res9

infinite_loop:
    beq $zero, $zero, infinite_loop
```


In-System Memory Content Editor - C:/hardware lab/lab 5 run 2/Quartus/lab5 - lab5
File Edit View Processing Tools Window Help
Instance Manager: Ready to acquire
Index Instance ID Status Width Depth Type Mode
0 ITCM Not running 32 256 RAM/ROM Read/Write
1 DTCM Not running 32 256 RAM/ROM Read/Write
JTAG Chain Configuration: JTAG ready
Hardware: DE-SoC [USB-1]
Device: @2: 5CSEBA6(JES)/5
Scan Chain
Instance 0: ITCM
Instance 1: DTCM
Show hidden icons

:Signal Tap


Signal Tap Logic Analyzer - C:/hardware lab/lab 5 run 2/Quartus/lab5 - lab5 - [stp6.stp]*
File Edit View Processing Tools Window Help
Instance Manager: Ready to acquire
Instance Status Enabled LEs: 3546 Memory: 274432 Small: 0/0 Medium: 27/553 Large: 0/0
auto_signaltap_0 Not running 3546 cells 274432 bits 0 blocks 27 blocks 0 blocks
JTAG Chain Configuration: JTAG ready
Hardware: DE-SoC [USB-1]
Device: @2: 5CSEBA6(JES)/5CS
Scan Chain
SOF Manager:
log: Trig @ 2025/06/30 16:43:54 (0:0:3.5 elapsed)
click to insert time bar
Type Alias Name
rst_i
STRIGGER_o
BPADDR_I[7..0]
CLKCNT_o[15..0]
EXinstruction_o[31..0]
EXpc_o[9..0]
FHCNT_o[7..0]
IDinstruction_o[31..0]
IDpc_o[9..0]
IFinstruction_o[31..0]
IFpc_o[9..0]
INSTCNT_o[15..0]
MEMinstruction_o[31..0]
MEMpc_o[9..0]
STCNT_o[7..0]
WBinstruction_o[31..0]
WBpc_o[9..0]
Data
Setup
Hierarchy Display:
MIPS
Data Log:
auto_signaltap_0